Klaus Kohl-Schöpe

Forth-Systeme unter Claude entwickeln

Bisher hatte ich meine Forth-Systeme mit MASM (für den Simulator) und eigenem Forth-Assembler/Compiler erstellt. Jedoch entkomme ich beruflich nicht dem KI-Boom und mein Sohn setzt Claude für seine Softwareentwicklung ein. Deshalb wollte ich auch wissen, wie einfach die Realisierung eines Forth-Systems und einer Applikation unter Claude ist.

Der erste Versuch

Nach Anmeldung (freie Version) habe ich folgende Aufgabe eingegeben:

Schreibe mir einen einfachen Forth Simulator in C

Herausgekommen ist das Programm forth.c mit ca. 350 Zeilen und einer Anleitung zur Compilierung.

```
gcc -std=gnu99 -Wall -Wextra -o forth forth.c
./forth # interaktiv
./forth datei.fs # Datei ausführen
```

Es wurde auch angegeben, welche Befehle schon in der ersten Version vorhanden waren:

- **Arithmetik:** + - * / mod negate abs min max
- **Stack:** dup drop swap over rot nip 2dup depth
- **Vergleiche und Logik:** = <> < > 0= and or xor invert, wahr ist -1, falsch ist 0
- **Ausgabe:** . .s cr emit ." text"
- **Eigene Wörter:** : name ... ;, Rekursion über recurse
- **Kontrollfluss:** if/else/then, begin/until, begin/while/repeat, do/loop mit i
- **Sonstiges:** (Kommentar), \ Zeilenkommentar, words, bye

Und auch ein Beispiel wurde angegeben:

```
: fak dup 1 > if dup 1 - recurse * then ;
10 fak . \ 3628800
```

Aber auch Einschränkungen gibt es:

- Keine `variable`, `constant` oder `create`.
- Die Kontrollstruktur-Paare (z. B. `if` und `then`) werden nicht auf passenden Typ geprüft.
- Es gibt keinen Heap-Speicher für Forth-Programme, und die Zellen sind `long`.
- `unistd.h` für `isatty` setzt Linux oder macOS voraus. Unter Windows brauchst du `io.h` und `_isatty`.

Ich habe mir gleich das Programm angesehen und gefreut, dass auch Fehlerbehandlungen vorhanden war und die Struktur soweit durchschaubar ist, dass man eigene Befehle ergänzen könnte (was ich auch später beim Debuggen machte).

Natürlich musste ich das Programm gleich kompilieren und im Windows-Terminal starten (in **fett** meine Eingaben):

```
Mini-Forth. 'words' zeigt alle Wörter, 'bye' beendet.
words
+ - * / mod negate abs min max dup drop swap over rot nip 2dup depth = <> < > 0= and or xor invert . .s cr emit i words bye
 ok
: fak dup 1 > if dup 1 - recurse * then ;
 ok
10 fak .
3628800  ok
```

Dies war schon mal eine positive Erfahrung und ich wollte wissen, wie schnell dieses Forth-System tatsächlich ist.

```
: x 0 DO LOOP ;
 ok
1000000000 x
 ok
```

Ist nicht nötig, die 0‘en nachzuzählen. Es waren 9 Stück und damit ein Schleifenwert von eine Milliarde und dafür hat mein ca. 2.5GHz schneller Prozessor nur etwas mehr als 3 Sekunden benötigt. Da kann doch was nicht stimmen, war mein erster Gedanke. Aber folgender Test, der dann ca. 8,5 Sekunden benötigt hat, bestätigte mir den Wert.

```
: z 0 swap 0 DO 1 + LOOP . ;
 ok
1000000000 z
1000000000  ok
```

Nebenbei hatte ich positiv aufgenommen, dass der Zeileneditor einen Speicher der letzten Befehle nutzt und dass man einfach durch Anhängen des Filenamens an `forth.exe` ein Programm nachladen kann.

Life und der erste Rückschlag

Nach diesen ermutigenden ersten Erfahrung wollte ich wissen, wie dieses Forth sich bei einem einfachen Programm schlägt, wobei dann auch einige der bisher fehlenden Wörter ergänzt werden sollten. Dafür habe ich Conway’s Life ausgewählt:

Schreibe ein Conway Life Forth Programm für diesen Simulator mit einem 25x25 Feld

Dabei stellt Caude fest, dass die benötigten 625 Zellen nicht auf dem Stack passen und er hat das Forth (umbenannt von mir auf forth2.c) um einige Wörter ergänzt:

```
variable constant create allot @ ! +! j …
```

Dazu lieferte er auch das Life-Programm (life.fs) dass diese 25x25 Felder umlaufend (als Torus) verwaltet und auch zugehörige Information zum Aufbau und drei Beispiel-Muster (Gleiter, Blinker und R-Pentomino) mit Anleitung sind vorhanden. Also versuchte ich gleich das erste Beispiel.

```
clear glider 60 run
```

Damit hatte ich den ersten Rückschlag, denn das Programm wurde ohne Fehlermeldung beendet.

Fehlersuche

Der große Vorteil von (auch unter C erstelltem) Forth ist die interaktive Entwicklungsumgebung, die ich hier natürlich nutzte, um die Ursache einzugrenzen:

```
forth2.exe life.fs
words
+ - * / mod negate abs min max dup drop swap
over rot nip 2dup depth = <> < > 0= and or xor
invert . .s cr emit i words j 1+ 1- <= >= 0< @
! +! allot here cells space spaces page ms bye
w h size grid next cx cy idx wrapx wrapy at set
clear population n@ neighbors rule step commit
generation show run glider blinker rpent
clear
 ok
glider
 ok
show
```

Also die doppelt verschachtelte Schleife oder die Ausgabe scheint das Problem zu sein, was sich schnell testen lässt:

```
forth2.exe life.fs
: x 10 0 do 10 0 do j 10 * i + . loop cr loop ;
 ok
x
0 1 2 3 4 5 6 7 8 9
10 11 12 13 14 15 16 17 18 19
…
90 91 92 93 94 95 96 97 98 99
 ok
```

Also an den Schleifen kann es nicht liegen, deshalb gleich der nächste Test mit der Ausgabe.

```
." Test"
Test ok
```

Die interaktive Ausgabe eines Strings geht noch, also weiter zum Kompilieren:

```
: Test ." Test" ;
 ok
Test
```

Da das Programm wieder beendet wurde, hatte ich den Fehler hiermit auf Kompilierung und Ausführung von `."` eingegrenzt.

Aufbau des KI-Forth

Um jetzt hier weiterzukommen, muss man erst mal verstehen, was Claude hier realisiert hat. Zuerst sieht man, dass er für die beiden Stacks je 256 Einträge reserviert. Auch die notwendigen Grundbefehle von Stackbehandlung bis zu den üblichen oben aufgelisteten Wörter sind in C realisiert.

Es gibt eine Tabelle für maximal 512 Befehle, die aus einem maximal 31 Zeichen langen Namen (längere werden einfach abgeschnitten), einem Pointer zu der C-Routine (oder 0), einem Pointer zum Programm (z.B. bei :-Definitionen) und der Anzahl der Befehle in der :-Definition besteht.

Für die oben gelisteten „Primitives“ werden beim Start des Programmes der Befehlsname und der Pointer zur C-Routine in die Liste eingetragen und kann mit Words angezeigt werden.

Was aber dann passiert, strapaziert schon etwas den Speicher, weil für jeden neuen `:`-Befehl zuerst in einer ebenfalls bis zu 512 Einträge großen Tabelle jeweils Befehlstyp und Parameter geschrieben und dann bei `;` in einen mit `malloc` reservierten RAM-Bereich kopiert werden. Der Name der :-Definition zusammen mit dieser Speicheradresse kommt anschließend in die Befehlsliste. Bei den Befehlstypen gibt es aktuell nur 7 Stück:

```
enum {OP_LIT, OP_CALL, OP_JMP,
OP_JZ, OP_DO OP_LOOP, OP_STR};
```

Die Parameter können Literals, Nummer des Befehls, Offset im Befehl oder die Stringadresse sein.

Fehlersuche – Teil 2

Mit diesem Wissen habe ich direkt im Sourcecode einige Funktionen wie Ausgaben des Strings während Kompilierung, `SEE` und `DUMP` ergänzt. . Mein erstes Test-

programm oben generiert mir folgende Einträge (hier mit Kommentar):

```
: z 0 swap 0 DO 1 + LOOP . ;
 ok
see z
Num: 50, Prim: 0, Instr: 1623274773152, Len: 8
 0 - 0            ( #0: 0 = Literal 0 )
 1 - 11           ( #1: 1 = Call - 11 = SWAP )
 0 - 0            ( #2: 0 = Literal 0 )
 4 - 0            ( #3: 4 = DO )
 0 - 1            ( #4: 0 = Literal 1 )
 1 - 0            ( #5: 1 = Call - 1 = + )
 5 - 4            ( #6: 5 = LOOP - Offset #4 )
 1 - 26           ( #7: 1 = Call - 26 = . )
 ok
```

Nach vielen Versuchen mit Abstürzen ist mir aufgefallen, dass bei einem 64-Bit Windows der `malloc` einen `void`-Pointer mit nicht nur 32-Bit (also `long`), sondern 64-Bit (`long long`) zurückliefert. Bei der Verwaltung der Adressen für das Vokabular wurden intern auch ein `void`-Pointer verwendet und damit die richtige Adresslänge verwendet. Die Befehlsaufrufe erfolgen auch indirekt über diese Pointer. Aber es handelt sich hier um ein 32-Bit FORTH und damit wird bei den Argumenten 32-Bit statt den für diese Stringadresse notwendigen 64-Bit gespeichert.

```
see Test
Num: 50, Prim: 0, Instr: 2540680010368, Len: 1
 6 - 2354320544   (ist nur ein 32-Bit Pointer)
```

Dies führte zum Absturz des Programmes in der alten Version mit 32-Bit. Nach der schnell zu realisierenden Umstellung der Datengröße auch auf 64-Bit (einfach `long` durch `long long` ersetzen) sieht es schon besser aus (Programmname auf forth2x.fs geändert).

```
see Test
Num: 50, Prim: 0, Instr: 2628889035984, Len: 1
 6 - 2628889041568
```

Und jetzt fliegt der Gleiter auch durch das 25x25 große Feld, wobei er ohne die 120ms Verzögerung für 1000 Schleifen weniger als 30 Sekunden brauchen und ohne Ausgabe noch fast 10-mal schneller ist.

Korrektur durch Claude

Ich wollte auch Claude die Möglichkeit zur Korrektur geben und habe folgendes gefragt:

Bei dem oben angegebenen forth.c und life.fs gibt es einen Absturz, weil die Adresse der für den String reservierten Speicher nicht in 64-bit im Programm abgelegt werden kann. Kannst du dies korrigieren ?

Die Antwort darauf war:

Das ist der Fehler in OP_STR: Ich habe den String-Zeiger in ein cell (long) gepackt. Unter Windows (LLP64) ist long nur 32 Bit breit, der 64-Bit-Zeiger wird abgeschnitten und fputs stürzt ab. Ich reproduziere das zuerst mit einem 32-Bit-cell.

Dies wurde in der korrigierten Version (forth3.c) dann so geändert, dass eine Tabelle mit 64-Bit-Pointer zum String verwendet wird. Auch eine mögliche Umstellung auf 64-Bit wurde erwähnt.

Schlussgedanken

Trotzdem bleiben diverse Wünsche offen, die vermutlich noch viele Rückfragen an Claude verursachen. Neben den noch fehlenden Befehlen ist vor allem das Speichern der Applikation als fertiges Programm wünschenswert. Was ich auch eliminieren würde, ist der Einsatz von malloc ohne free. Zum Beispiel könnte man einen größeren Speicherbereich fest bis zum Ende der Laufzeit reservieren und Programm und Daten dort ablegen. Dies dürfte wenig an der Geschwindigkeit ändern und das Speichern einer Applikation erst möglich machen.

Wie immer bei automatischen Tools müssen diese mit Köpfchen verwendet werden. Für Tests muss die Struktur des erzeugten Programms verstanden und jedes Wort sollte sowohl interaktiv als auch in anderen Wörtern getestet werden. Dafür erhält man mit wenig Aufwand ein sehr schnelles Forth, dass auch sehr leicht auf Linux und Microcontroller portiert werden kann.

Bitte auch daran denken, dass außer der Unsicherheit zu dem Recht an KI-Inhalten es in letzter Zeit auch noch einige Urteile gab, welche die Nutzung von KI für Aufgaben wie z.B. Gutachten oder Softwareentwicklung nicht als echte Dienstleistung anerkannt hat und den Kunden die Kosten rückerstattet werden mussten.

Trotzdem viel Erfolg bei euren Versuchen.

Klaus Kohl-Schöpe